
PRACTICAL EXAM ON PARALLEL PROGRAMMING

Telecom Nancy – Université Lorraine, 2025 - 2026

Upload deadline 15:30



At the end of the session, you must **upload your archive on Arche** that contains your **source files** as well as a **text file** with **your results**.
The upload will be definitively closed at **15h35**.



The archive provided in the practical exam section of the [Arche website](#) contains the following files :

- `CMakeLists.txt` : automatic configuration and compilation file
- `debug_MPI.h` : header file containing additional debugging information for MPI
- `search.c` : source file of the program to complete

The search program finds the positions of the **first** and **last** occurrences of a given pattern in a text sequence composed of the letters A,C,T,G (like DNA sequences) and provides its **total number** of occurrences. The final result is stored in the `Result` data structure defined at the beginning of the source file.

The program generates a pseudo-random sequence of given size (option `-n`). The pseudo-random seed can be specified with the option `-s`. The pattern to search is specified with option `-p`. In this program, a sequential version and a parallel version are included. Times and parallel performance are given at the end of the execution.

However, the parallel version is not operational and must be completed. The objective is to obtain a parallel version combining MPI processes with `OpenMP` threads. To achieve this goal, the three following steps are proposed :

- `OpenMP` alone (one MPI process)
- MPI alone (one `OpenMP` thread per MPI process)
- Combination of both parts

1 OpenMP alone

In this part, we consider the computation with only one MPI process using a specified number of `OpenMP` threads (option `-t`).

In the current version of the source file, the function `search_mono(...)` performs a sequential search of the pattern in the text.

a) Modify this function in order to use the given number of `OpenMP` threads *only* if more than one thread is specified. This means that no `OpenMP` thread must be created if the number of threads is one.

b) In order to check the validity of your modification, execute the program with the pattern `ATGCTCGA`, 2 threads and a seed value of 1. You should obtain identical results between the two versions and a smaller execution time for the parallel version. Add the trace of the execution to the text file of your results.

c) Once your program is valid, execute the program with 2, 3 and 4 threads and copy the traces to your results file.

